

Time = 2° Packageage / Link rate transmission

Part1: Time = (N + M) L/R, SAF transmission = Packet switch must receive entire packet before starting transmission of packet onto outbound link.**Packet Switching: Output buffer/queue** 1temp stores packets when link is busy => 2queuing delays, Link full => 3packet loss. **Forwarding tables - Routing protocols:** determines how packets are forwarded by routers, each host identified by IP, **Tables** maps dest ip to output link, **Protocols** configure the tables. **Circuit switching:** Establishes connection before sending and guaranteed constant rate, Resources needed along the path, communication between the end systems are reserved for the duration of the communication session. offers predictable end-end delays (no queuing), suited for real-time services. **Packet switching:** Better sharing of transmission capacity, simpler/more efficient/less costly to implement. **Multiplexing multiple circuits over 1 link: Frequency Division Multiplexing (FDM):** Each frequency band is dedicated to a specific sender-receiver pair. **Time Division Multiplexing (TDM):** All slots with the same label dedicated to a specific sender-rec. Pair, all connections get same frequency but different time. **ISPs: tier 1 global transit ISPs, t2 regional ISP, access ISP, internet contains points of presence (PoPs),** multi-homing, peering, and Internet exchange points (IXPs). **Pop** = group of one or more routers where customer ips connect into the provider isp. **multi-home** to: connect to two or more provider ISPs. To reduce costs, a pair of **nearby same level ISPs** can **peer**: Connect to each other to avoid paying the upstream isp for their service. **third-party company** can create **Internet Exchange Point (IXP)**: Meeting point where multiple ISPs can peer together. **Nodal delay: Processing delay:** time required to examine packet's header and determine where to direct it (+ check for bit-level errors). **Queueing delay:** time the packet spends waiting in buffer to be transmitted (depends on nr. of earlier-arriving packets waiting). **Transmission delay:** time to transmit package on the link (depends on L and R). **Propagation delay:** time required for bits to propagate from src to dest router of the link (limited by speed of light). Prop. delay = d/s, with d = link distance. **Throughput: Instantaneous throughput:** The rate (in bits/Sec) at which a host is receiving a file at a specific time. **Average throughput:** The average rate at which an entire file is downloaded. File of F bits and transfer takes T seconds, the average throughput is F/T bits/sec. **Bottleneck link:** The link on the path from source to destination with the lowest transmission rate. Throughput depends on transmission rate and co-existing traffic **Throughput** depends on **transmission rate** and **co-existing traffic**. **Protocol:** a protocol defines the format and the order of messages exchanged between 2 or more communicating entities as well as the actions taken on the transmission and/or receipt of a message or other event. **App. lay** (message): distributed over multiple end-systems. Consists of the applications and their associated app.-layer protocols (HTTP, SMTP, FTP, DNS). **Transport lay** (segment): transports app.-layer message between end-systems. **TCP:** Connection-oriented, reliable (retransmits lost packets), flow control (sender/receiver speed matching), congestion control (throttles transmission. Rate to avoid network congestion). **UDP:** Connection-less, unreliable, no flow control, no cong. control. (low overhead and faster than TCP) **Network lay** (datagram): responsible for moving network-layer packet from host to host. **funcn:** Addressing and Routing (intra- and inter-domain routing protocols). **Link lay:** transfers data (called frames) across link. **funcn:** depends on linklayer protocol: Reliability, error detection, addressing and media access. ethernet, wifi, docsis, ppp **Physical lay.:** Transfers the individual Bits over the medium(fiber, optical, radio). **funcn:** Modulation, coding and bit synchronization. Depends on the medium. **Encapsulation:** Layers encapsulate the message in their own header and pass it on, they only need to implement the layers they are active in. router implements network, link, physical layer. **Network performance:** characterized by delay, packet loss and throughput. **Part2: Transport layer** could offer to applications: **Reliable data transfer:** Ensures that data sent by one process will arrive at the receiving process **Throughput:** Guarantees minimum available throughput at a specified rate (in bits/second) **Timing:** Guarantees a maximum end-to-end delay for transmitted data (in seconds) **Security:** Ensures security and privacy (e.g., confidentiality, integrity, authentication) of data. **Transport Layer Protocol (TCP): Connection-oriented:** Handshake. **Reliable data transfer service:** All data is guaranteed to be delivered in order and without error or loss. **Congestion control:** Throttles sender to avoid network congestion. **Fairly share** available bandwidth. **Security:** Confidentiality, integrity and host authentication are offered through the TLS protocol. **User Datagram Protocol (UDP) Connectionless:** No handshake. **Unreliable data transfer:** Data may not arrive out of order. **No congestion control:** UDP processes can send as much data as they want/can. The Internet's transport layer does not offer throughput or timing guarantees! **Application-layer protocol** defines how an application's processes, running on different end systems, pass messages to each other. The **HTTP protocol** defines the rules of communication between web clients (browsers) and servers. **HTTP: Stateless:** Server sends requested files to clients without storing any state information about the client. **Non-persistent (HTTP1)** connections = new TCP con. for each object (2RTT/object). **persistent connections (HTTP1.1)** = one TCP connection for all objects (1RTT + 1RTT/object). **HTTP 1.1 Pipelining** not broadly used instead **multiplexing via HTTP/2** is used. **Request methods: GET** Retrieves the information identified by the request URL, May be used to pass parameters via the URL. **HEAD** identical to GET, except that the server will not return the message body in the response. **POST** Used to pass parameters to server via request body. Returned information will depend on the request URL and the passed parameter values. **PUT** Allows the user to upload an object (provided in the request body) to a specific path (specified by the URL) on the web server **DELETE** Allows the user, or an application, to delete an object (specified by the URL) on a web server. **Cookies** can be used to identify a user. **Web cache/ proxy server** = Network entity that satisfies HTTP requests for an origin Web server. The Web cache has disk storage and keeps copies of recently requested objects in this storage. **Caching advantages:** Reduces response time, avoids bottleneck links, reduce traffic access link to the internet. Request without cookies -> response with set-cookie Request with cookie -> cookie specific action (response)

requesting host -> local DNS -> root DNS -> local -> com TDL -> local -> Auth DNS -> Local -> host

Conditional GET if modified since (date). Single TCP connection for all objects of page has a **Head of Line (HOL)** blocking problem meaning large objects in front of other objects will block them. **HTTP/2** solves this by **splitting** a message in frames and **interleaving** the frames, smaller objects behind large objects will arrive quicker. **HTTP/2 features: Message Prioritization (HTTP/2)** 1allows web developers to customize relative priority of requests. 2Each request is assigned a weight between 1 (lowest) and 256 (highest). 3The server first sends the frames of responses with highest priority (interleaving those with equal priority). 4Client can define dependencies between requests. **Server Push** 1allows the server to send multiple responses to a single request. 2Useful to pro-actively send additional content that is part of a requested HTML base page. 3Can also be used for pushing segmented video. 4Eliminates the extra latency due to waiting for multiple requests. **SMTP** agentA sends mail -> A's mail Server -> agentB's mail server -> agentB (PC, phone) **DNS: convert** hostnames to IP, **consists of:** 1distributed hierarchy of dns servers and 2app layer protocol that allows hosts to query database. Uses **UDP port 53**. **Host aliasing:** Defines alternative hostnames in addition to the main canonical hostname. **Mail server aliasing:** Similar to host aliasing but for email addresses. **Load distribution:** Associating multiple IP addresses with 1 alias hostname to spread out traffic. **Root DNS servers** provide the IP addresses of the TLD servers. **TLD servers** provide the IP addresses for authoritative DNS servers. For each of the top-level domains there is one TLD/cluster. Every organization with publicly accessible hosts (such as Web servers and mail servers) on the Internet must provide **publicly accessible DNS records** that map the names of those hosts to IP addresses. An organization's **authoritative DNS** server houses these DNS records. **Local DNS server** acts like a nearby dns proxy and cache. **DNS resource records (RRs)** = (Name, Value, Type, TTL): Name and Value depend on type. TTL decides when cache needs clearing. **Types: A:** Provides a standard hostname-to-IP address mapping (Name is hostname and Value is IP address) **NS:** Used to route queries along the query chain (Name is domain and Value is authoritative DNS server) **CNAME:** Maps alias hostnames (Name) to canonical hostnames (Value) **MX:** Maps alias mail server names (Name) to canonical mail server names (Value). **Part3: Transport layer error checking:** check the integrity of received segments using error-detection header field. **Multiplexing:** Encapsulating data from different sockets into segments and passing them to the network layer. **Demultiplexing:** Identifying the receiving socket based on header fields and directing the data to that socket. Identification of the correct socket (and thus process) is done based on source and destination port number. a **UDP socket** is fully identified by a two-tuple consisting of a destination IP address and a destination port number. When TCP segment arrives at the host, all four fields (source IP address, source port, destination IP address, destination port) are used to direct (demultiplex) the segment to the appropriate socket. **why udp ipv tcp:** 1Finer application-level control over what data is sent, and when. 2No connection establishment. 3No connection state (i.e., less overhead per connection on the server) 4Smaller packet header overhead (i.e., 8 bytes for UDP vs. 20 bytes for TCP). **UDP checksum: Sender:** Calculated by the sender as the 1s compl. of sum of all 16-bit words in the UDP segment (+ pseudo header) Overflowing bits are wrapped around. **Receiver:** Add all 16-bit words of the segment together with the checksum. Result should be 11111111 11111111. If an error is detected (there will be a 0), the packet is usually discarded. **ARQ** protocols is based on 3 fundamental principles. **Error detection:** use checksum for errors **Receiver feedback:** ACKs|NAKs. **retransmission** if no ack, timeout or errors. **Stop-and-wait fundamental problems:** if an ACK is **corrupt**, the packet can be resent. 1This introduces duplicate packets 2The receiver needs a way to identify retransmission packets. 3This can be done by adding sequence numbers (stop-and-wait only requires 1-bit sequence number! (known as alternating-bit protocol)). Sender can detect a **lost** packet or ACK by using a timeout. 1If no response is received after the timeout, either the packet or ACK was lost in transmission 2Timeout should be at least the duration of a worst-case round-trip-time (RTT). 3Worst-case RTT is very hard to estimate in most real networks. **Go-Back-N: Sender:** Can send from base to base+window_size(N) **Cumulative acks:** Ack for packet with seq numb n -> all packets with seq numb <= n arrived, move window on new ackn. **Timer:** retransmit all unack packets on timeout. **Receiver** has field expectedseqnum. when packet with exp numb is received, the exp numb is increased. if a packet is out of order, the packet is discarded and an ACK with the highest in-order sequence number is sent. **SRsender:** Data received from the application layer is only transmitted if the next available sequence number falls within the sender's window. A logical timer is maintained for each packet, since only a single packet is retransmitted on a timeout. If an ACK is received, the sender marks only that packet as received. If the sequence number equals send_base, the window moves to the next unacknowledged packet. Untransmitted packets that fall within the new window will then be transmitted. **SR receiver:** If a packet arrives within receive window, a selective ACK is returned and it is buffered. If seq nr equals rcv_base, then it and all consecutively numbered buffered packets are delivered to the upper layer, and the rec. window is moved forward by the number of delivered packets. If a packet with sequence number [rcv_base-N, rcv_base-1] is received, an ACK is generated. Otherwise: Discard the packet. The window size N must be <= half the seq nr space. This is required in the corner case where all N ACKs of sender window packets are lost. Otherwise it might not be possible to differentiate between a new or retransmitted packet. **TCP** connection provides **full-duplex** service: If there's TCP connection between Process A on one host and Process B on another host, then app-layer data can flow from Process A to Process B at the same time as app-layer data flows from Process B to Process A. TCP connection is also always **point-to-point** = between a single sender and a single receiver=> multicasting not possible. **TCP** views data as unstructured, but ordered, stream of bytes seq nr of a segment refer to the position of its first byte in the stream and NOT the number of the segment itself. Acknowledgement number is the next byte that receiver expects from the sender. TCP thus uses **cumulative acknowledgments**.

Estimating RTT: The TCP retransmit timeout should be larger than the RTT. How to estimate the RTT in a real network? 1Measured as time between sending segment and receiving the corresponding acknowledgment (SampleRTT). 2Never calculated for segments that are retransmitted. 3To reduce fluctuations, EstimatedRTT is an exponential weighted moving average (EWMA) of SampleRTT. 4The variability of the RTT is calculated as DevRTT. **EstimatedRTT** = (1-α)* EstimatedRTT + α*SampleRTT. how much sampleRTT afwijkt = **DevRTT** = (1-β)*DevRTT + β*[SampleRTT - EstimatedRTT] (alpha=0.125, beta = 0.25). **TCP retransmission timeout interval:** TimeoutInterval should be close to but larger than estimatedRTT. 1TimeoutInterval = EstimatedRTT+4*DevRTT (when estimatedRTT updated). 2An initial TimeoutInterval value of 1 second is recommended. 3If timeout occurs => TimeoutInterval = TimeoutInterval*2. 4Margin should be larger if the variability (DevRTT) is large. **TCP sender (FSM) steps:** 1Data received from app above 2Create TCP seg with seq nr NextSeqNum. 3Start retransmission timer (if not running yet). 4Pass seg to IP. 5NextSeqNum = NextSeqNum + len(data). if Timeout occurs: retransmit oldest unacknowledged segment. Restart timer. 1ACK(y) received with y > sendBase: SendBase = y. 2Restart timer (if needed), sendBase = seq of oldest unACKed byte. **TCP receiver:** 1Arrival of in-order segment: if next in-order segment doesn't arrive (after 500ms), send ACK. 2Arrival of in-order seg, with another in-order seg waiting for ACK: send single cumulative ACK for both seg. 3Arrival of out-of-order seg with higher-than-expected seq nr: send duplicate ACK with seq nr of first expected in-order byte. 4Arrival of in-order segment, when gap exists: immediately send ACK. **Fast retransmission:** 1Timeout period in TCP can become very long => results in delays when retransmitting lost packets. 2Lost packets can also be detected by counting duplicate ACKs (occur when out of order segments arrive). 3After 3 duplicate ACKs for the same data oldest non-acknowledged segment is retransmitted (sender does not wait for the timer to expire). **TCP error recovery** is a hybrid approach, combining features of Go-Back-N and Selective Repeat. **GBN similarities:** 1TCP uses cum. ack. 2Out-of-order segments are not individually ACKed. 3TCP sender maintains seq nr of smallest transmitted but unacknowledged byte (SendBase) and next byte to be sent (NextSeqNum). **GBN differences:** 1Most TCP implementations buffer out-of-order received segments. 2TCP does not retransmit all unacknowledged segments if an acknowledgement is still received before next timeout. **SR Similarities:** Proposed TCP modification for selective acknowledgment allows TCP to acknowledge out-of-order segments selectively. **Flow control:** Ensures that sender doesn't send data faster than receiver application reads it. **Receiver:** 1Receive buffer (LastByteRcvd - LastByteRead <= RcvBuffer) = data in buffer (has LastByteRead en LastByteRcvd) + spareroom. 2Receive window (rwnd) = RcvBuffer - (LastByteRcvd - LastByteRead). **Sender:** LastByteSent - LastByteAcked (=amount of unACKed data) <= rwnd. (Feedback from receiver via header field: rwnd). **TCP cong control:** when network is overloaded, segments get lost. 2 ways to reduce sending rate: **end-to-end cong. control:** 1Presence of congestion is inferred by end systems based on observed network behavior. 2Approach used by TCP. 1aTCP packet loss used to identify congestion. 1bTCP decreases window size accordingly. **Network-assisted cong. control:** 1Routers provide explicit feedback to the sender and/or receiver regarding congestion state. 2single bit or more detailed feedback. 3Supported by TCP as an option. **TCP congestion-control algorithm: Slow start:** 1cwnd(congestion window) is initialized to 1 MSS and increased by 1 MSS when a transmitted segment is first acknowledged. (1-2-4-8-...). 2When a timeout occurs: ssthresh (slow start threshold) is set to cwnd/2. cwnd is reset to 1 MSS. 3When cwnd ≥ ssthresh TCP transitions into congestion avoidance mode. 4When 3 duplicate ACKs are received TCP transitions into fast recovery state. **Congestion avoidance:** 1Starts when cwnd is approximately half its value when congestion was last encountered. 2TCP only increases cwnd by 1 MSS every RTT (instead of doubling it). Common approach: When new ACK arrives, cwnd = cwnd + MSS^2/cwnd. 3TCP goes back to slow start state when a timeout occurs and to fast recovery after 3 duplicate ACKs. **Fast recovery:** 1Starts when 3 duplicate ACKs are received during slow start or congestion avoidance. ssthresh is set to cwnd/2. cwnd = ssthresh + 3MSS. 2When timeout occurs, TCP goes to slow start (setting cwnd to 1 MSS and ssthresh to cwnd/2) 3When ACK arrives for the missing seg, TCP goes back to congestion avoidance (resetting cwnd to ssthresh) 4TCP Tahoe does not use fast recovery. 5The new TCP Reno incorporates fast recovery. **TCP Reno** 1Additive-increase, multiplicative-decrease (AIMD). 2Slowly increases cwnd after packet loss. **TCP CUBIC** 1Quickly ramps up sending rate to get close to preloss sending rate. 2Probes cautiously afterwards. 3Only changes congestion avoidance phase. **Explicit congestion notification (ECN):** 1Uses 2 bits in the IP header. 1aECT bit is set to 1 if sender and receiver support ECN. 1bECE bit is set to 1 if an intermediary router experiences congestion. 2If the EC bit is 1 on a received TCP packet, the receiver will set the TCP ECE bit to 1 in its ACK. 3If the sender receives an ACK with ECE=1 then it acts as if a packet was lost and sets CWR=1 in next segment. **fairness** 1A cong. control mechanism is **fair** if K connections sharing a bottleneck link with transmission rate R bps, each achieve an average transmission rate of approximately R/K. The TCP AIMD algorithm has been shown to be fair. **Part4: Forwarding (data plane):** 1Move packet from the router's input link to the appropriate output link 2Local decision taken at very short time frame. 3Typically implemented in hardware. **Routing (control plane):** 1Determine the route or path taken by packets from sender to receiver. 2Network-wide process that takes place on longer time scales. 3Typically implemented in software. **Traditional control plane:** router has local forwarding table (Header -> Output). **SDN control plane:** uses physically separate, remote controller that computes and distributes the forwarding tables to be used by each router. **Destination-based forwarding:** 1The forwarding table cannot contain an entry for each destination address. 2Instead, forwarding table entries consist of ranges, represented by prefixes. 3If a packet matches multiple prefixes, **longest prefix matching** is used. **FSM** each underlined part is a loop

DHCP: Dynamic host configuration protocol, DNS: domain name system, SMTP: simple mail transfer protocol, FTP: file transfer protocol, HTTP: hypertext transfer protocol

ECT: ECN-capable-transport, CE: congestion experienced, SDN: Software defined networking, MSS: maximum segment size, ARQ: Automatic repeat request, SR: selective repeat

N: set of all nodes; **D(v):** cost of the currently known least-cost path from the source to v.

Queueing in routers: When input or output buffers become full, packet loss will occur. **HOL** blocking occurs when a packet in input queue must wait because it is blocked by another packet at the head of the line. **Output port contention** occurs when multiple packets are destined for the same output line at the same time. **CIDR** determines the Internet's address assignment strategy. **2**Each 32-bit IP address is split in two parts (a.b.c.d/x) (x indicates the number of bits in the first part, also referred to as subnet). A **DHCP server** offers several functions to hosts within an organisation (or subnet): **1**Automatically assign IP addresses to hosts. **2**Provide additional information (subnet mask, IP address of first-hop router, local DNS server). **DHCP consists of 4 steps:** Discovery, Service offer(s), Request, Acknowledgment. Assigned IP address has lifetime that can be renewed.

Network Address Translation (NAT): **1**There are not enough IP addresses to give every host in every network a unique IP address. **2**NAT offers a solution by associating a set of private addresses with a single public IP address (basically using port numbers to address hosts). The approach to IPv4-to-IPv6 transition involves **tunnelling**: Suppose a datagram is sent from router A (ipV6) to F (ipV6) with C and D being ipV4 routers. The sending side of the tunnel(B) takes the entire ipV6 datagram and puts it in the datafield of an ipV4 datagram. all routers in the tunnel (C and D) will route the datagram like usual without knowing that there is an ipV6 datagram in the ipV4 datagram. **routing algorithm** determines good paths from sender to receiver through the network of nodes (N)(routers) and edges(E)(links). tries to find the least-cost path of edges $\{x_1, \dots, x_p\}$ between source node x_1 and destination x_p . **(centralized) link-state routing algo:** **1**Can apply known least-cost path algorithms (Dijkstra algorithm). **2**Notations **N:** set of all nodes, **s:** source node, **D(v):** cost of the currently known least-cost path from the source node s to destination v. **p(v):** previous node along the current least-cost path from the source s to v. **N':** subset of nodes for which the least-cost path from the source s is known. **Dijkstra's Algorithm: Initialization:** $N' = \{s\}$, $\forall v \in N: D(v) = c(s, v)$. **Repeat:** $w = \text{argmin } D(v)$, $N' = N' \cup \{w\}$, $\forall v$ neighbours of w of $w \wedge v \notin N'$: $D(v) = \min(D(v), D(w) + c(w, v))$ **Until:** $N' = N$. (this is $O(n^2)$ or $O(n \log n)$). **The distance vector routing algo:** **1**Distributed routing algorithm where each node only relies on information obtained from neighbours. **2**Relies on the Bellman-Ford equation: $d_s(y) = \min_v \{c(x, v) + d_v(y)\}$ where $d_v(y)$ is the least-cost distance from x to y. **3**The solution provides nodes x forwarding table. **4**Each node x keeps track of: **4a**An estimate distance vector $D_x = [D_x(y): \forall y \in N]$, **4b**The cost $c(x, v)$ to each of its direct neighbors v. **4c** the distance vector of each of its neighbors v, $D_v = [D_v(y): \forall y \in N]$. **Initialization:** $\forall y \in N: D_x(y) = c(x, y)$. $\forall$ neighbors w: send $D_x = [D_x(y): \forall y \in N]$ to w. **Repeat:** (whenever link cost changes, or distance vector received from neighbor) $\forall y \in N: D_x(y) = \min_v \{c(x, v) + D_v(y)\}$. if $D_x(y)$ changed for any destination y: $\forall$ neighbors w: send $D_x = [D_x(y): \forall y \in N]$ to w. **forever .** **count-to-infinity problem.** (routing loop when link cost increases) $\Rightarrow$ **solution: poisoned reverse:** if z routes through y to get to destination x, then z will advertise to y that its distance to x is infinity, that is, z will advertise to y that $D_z(x) = \text{inf}$. z will continue telling this lie to y as long as it routes to x via y. Since y believes that z has no path to x, y will never attempt to route to x via z, as long as z continues to route to x via y (and lies about doing so). **Routing on the Internet happens on two levels:** **1**Within an AS (intra-AS): Interior Gateway Protocol (IGP), such as Open Shortest Path First (OSPF). **2**Across ASs (inter-AS): Exterior Gateway Protocol (EGP), such as Border Gateway Protocol (BGP). **advantages of two-stage approach:** **1**Scalability: the number of routers on the internet is too big for a single routing algorithm to handle **2**Administrative autonomy: each IDP wants to enforce its own policies on the routing algorithms it uses. **Interior Gateway Protocols (IGPs): OSPF:** based on link-state routing+Dijkstra's algorithm. **1S-1S:** same as OSPF. **EIGRP:** Based on distance-vector routing. **OSPF:** **1**Routers broadcast link-state information to all other routers in the autonomous system, not only neighbors. **2**Link-state is broadcast when a link changes (cost change, comes up, or goes down). **3**Link-state is also broadcast periodically (at least once/30min). **4**Link availability checked by sending periodic HELLO messages to all direct neighbors. **5**Link weights configured by the administrator. **6**Security: Exchanges between OSPF routers can be authenticated. **7**Multi-path: if multiple same-cost paths exist, they can be used simultaneously to each carry part of traffic. **8**Multicast support. **9**Hierarchical division of AS into multiple independent routing areas. **BGP** **1**used for routing between all ASs on Internet. **2**Decentralized and asynchronous, based on distance-vector routing. **3**Routing is done based on CIDR prefixes (e.g., 138.16.68/22) instead of individual destination addresses. **4**Routers maintain semi-permanent TCP connection on port 179 to exchange BGP information. **5****BGP connection:** TCP connection between two routers within the same AS. **6****BGP connection:** TCP connection between two routers in different ASs. **7**Within an AS, all routers maintain an iBGP connection to each other (in a full mesh), while eBGP connections are only maintained between direct neighbors. **BGP advertisements** (or routes) consist of address prefix and several attributes **1**AS-PATH: List of ASs through which the message has passed. **2**NEXT-HOP: IP address of the router interface that begins the AS-PATH. content of BGP ADV: (NEXT-HOP=router on start of arrow; AS-PATH; PREFIX=subnet). **BGP** sequentially invokes elimination rules until only **1 route** remains: **1.** if any routes have the local preference value set, route with the highest loc pref is selected **2.** from remaining routes, the one with shortest AS-PATH is selected **3.** From remaining routes, the one with closest NEXT-HOP router is selected (determined using intra-AS routing **4.** If more than 1 route remains, a route is selected based on the BGP identifier. **IP-Anycast:** BGP can be used to advertise the same IP address for multiple servers, allowing routers to automatically select the nearest one (e.g., DNS server replication). **Generalized forwarding in OpenFlow (SDN):** **1**Flow table consists of 3 columns: **1.** Header field values to match (including link layer, network layer and transport layer fields). **2.** Counters keeping track of number of matches and time table entry was last updated. **3.** Actions to be taken (forwarding, load balancing, rewriting header fields (=Modify-field), dropping packets).

Matching is based on 3 packet-header fields and the incoming port ID. 3Header values can be wildcarded (e.g., 128.119.*.* matches all IP addresses starting with 128.119). 4Each flow table entry has a priority, using many more entries match with a single packet header. **SDN architecture:** **Data plane:** SDN controlled switches. **SDN control plane** divides into 2 components—the SDN controller and the SDN network-control applications. controller's functionality can be broadly organized into three layers: communication layer: communicating between the SDN controller and controlled network devices. network-wide state-management layer. interface to the network-control app layer. **Part 5: Link layer possible services:** **Framing:** encapsulate network-layer datagram in frame. **Link-Access:** Medium access control (MAC) protocol, specifies rules of frame transmissions. **Reliable delivery:** Error-free. **Error detection/correction.** **Parity checks:** 1single error detection bit where sum of all bits must be even. 2only detects when an odd num of errors occur. 3Bad in practice as bit errors do not happen independently. **Two-dimensional generalization parity scheme:** The d bits in D are divided in i rows and j columns. Parity bit is computed for each row and column. **Forward Error Correction (FEC) = Single bit errors** will show an error in both the row and column parity bit, and can thus be corrected. **Checksum:** 1s complement of the sum of all 16-bit words in the UDP/TCP segment (+ pseudo IP header). **Cyclic redundancy check (CRC) codes:** **polynomial arithmetic** 1Each bit string can be represented as a polynomial with binary coefficients. 2all calculations are done in modulo-2 arithmetic without carrier in addition or borrows in subtraction. 3addition and subtraction are thus identical and equal to the bitwise exclusive-OR (XOR operation) 4multiplication by 2^k can be calculated by left shifting a bit parent by k places. **Approach** 1sender and receiver agree on **generator G = r+1 bit pattern**. 2Sender appends r bits R to data D so that resulting **d+r bit** is divisible by **G**. 3Receiver divides d+r received bits by G, if nonzero remainder $\rightarrow$ error. **R=remainder (D^2^r/G).** **CRC standards:** 1Detect up to r-1 consecutive bit errors. 2Detect consecutive bit errors of length greater than r+1 with a probability $1-0.5^r$. 3Detect any odd number of bit errors. **multiple access protocol:** When many nodes transmit and receive data over a shared broadcast medium, a multiple access protocol is needed to avoid collisions. **Desires:** 1Full utilization of throughput R bps. 2Fair average throughput of R/M bps for M nodes. 3Decentralized, 4Simple and inexpensive to implement. **Channel partitioning protocols:** Frequency Division Multiple Access (FDMA), Time Division Multiple Access (TDMA), **Channel partitioning protocols(CDMA)**(check fdm and tdm **part1**) 1sender is assigned unique M-bit, code c and data d. 2 The code changes as a. much faster rate than the sequence of data bits 3 each transmitted bit is encoded by multiplying the code 4 for convenience, a value of -1 is used for 0 bits. for the k th databit d, and the m th mini-slot of the bit-transmission time of d, the output of the CDMA encoder $= Z_{km} = d_k \cdot c_m$. **Receiver:** arrives as sum $Z^*(i,m)$. Original databit of sender $d_i = (\text{sum}(m=1 \rightarrow M) c_m^*)/M$. **Assumptions:** 1CDMA codes must be carefully chosen. 2Received signal strength from senders is the same. 3Transmissions of different users are time-synchronized. **Random access protocols:** 1Each transmitting node transmits at the full rate of the channel. 2When there is a collision, nodes repeatedly retransmit their frame. 3Each node waits a random delay before retransmitting. 4Collisions are avoided if the random delays are different enough. **Slopped ALOHA:** After collision, node retransmits during each subsequent slot with a probability $p \in [0,1]$ til successful. Transmissions occur at the beginning of (synchronized) timeslot. Assuming a fixed frame length L bits, channel rate R, each slot takes L/R sec. **Assumptions:** Each node of the set of N nodes transmits during each slot with a probability p. $N \cdot p(1-p)^{N-1} = \text{prob}$ that any node transmits successfully, for a large number of senders, the maximum efficiency is $1/e \sim 0.37$. **Pure aloha:** 1Node immediately transmits a frame, instead of waiting for a start of the next time slot 2If a collision occurs, the node immediately retransmits with probability p. 3Otherwise, the node waits for a single frame transmission time and retransmits again with probability p. succ transmission probability: $p(1-p)^{2(N-1)}$, maximum efficiency is $1/2e \sim 0.18$. **Carrier Sense Multiple Access with Collision Detection (CSMA/CD):** **Carrier sensing:** listening to channel before transmitting and only trans if no other trans is ongoing. **Collision detection:** if trans is detected while transmitting, stop trans, and wait a random amount of time before retransmitting. **Random backoff time:** 1if backoff time is large and few nodes collide $\rightarrow$ channel will remain idle 2If backoff time is small and many nodes collide $\rightarrow$ collisions will keep occurring 3backoff should be long when many nodes collide and small when few nodes collide **Binary exponential backoff:** 1After n subsequent collisions, select backoff time K uniformly at random from $\{0,1,2,...,2^n - 1\}$. 2After a successful transmission, the value of n is set to 0 again. 3Specifically for Ethernet, the actual time equals $K \times 512$ bit times (i.e., K times amount of time needed to transmit 512 bits). The efficiency of ethernet is $1/(1+5^d)$ ($d_{\text{prop}}/d_{\text{trans}})$ with d_{prop} propagation delay / d_{trans} transmission time **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA):** used by wifi, since collision detection is harder due to transmission power being higher at the source and not all trans can be heard by all nodes. **Collision avoidance:** 1If a node has a frame to transmit and the channel is sensed busy, it immediately starts a random backoff counter (binary exponential backoff), 2While the channel is busy, the timer is frozen, 3When the counter reaches zero, the frame is transmitted fully and the node waits for an acknowledgment, 4If no acknowledgment is received within a timeout, the maximum backoff is increased and a new random backoff counter is started. **Random access protocols** do not possess the desirable property that when M nodes are active their average throughput should be close to R/Mbps which **Taking-turns protocols** do achieve. **Polling protocol:** Master node polls nodes in round-robin fashion (eg Bluetooth protocol) 1Advantages: Eliminates collisions and empty slots that occur in random access protocols 2Disadvantages: Polling causes delays, master node results in a single point-of-failure.

subset of nodes for which the least-cost path from the source s is known

Token-passing protocol: Only node holding token can transmit, passes token along sending maximum number of frames (or having no frames left to send) (eg fiber distributed data interface (FDDI), IEEE 802.5 token ring protocol). **1**Advantages: Eliminates collisions and empty slots

2Disadvantages: Complex failure cases (single node failure can crash the channel, recovering from lost tokens) **Local area network(LAN):** Switches operate link layer, don't recognize networklayer addresses, and don't use routing algorithms (like OSPF). **1**Addresses at link layer are called MAC addresses. **2**Each network adapter has unique MAC address. **3**Switches do not have MAC addresses (nor IP addresses). **4**MAC addresses(usually 6-byte) were designed to be permanent, but now they can be changed via software. **5**Each vendor of network adapters is assigned a unique 24-bit MAC address prefix by IEEE, to ensure all MAC addresses are unique. **ARP** provides MAC address associated with IP within the LAN. hosts and routers have **ARP table** to cache entries. Broadcasts ARP-req → ARP-resp from host with the ip → add entry in ARP table. **Switches:**

Filtering: Determine whether a frame should be forwarded to some interface or dropped.

Forwarding: Determine interfaces to which frame should be directed (switch table) **1**Dest addr not found: broadcast on all interfaces (except incoming interface). **2**Dest addr linked to incoming interface: discard frame **3**Dest addr linked to other interface: forward to that interface. **self-learning:** switch tables are built automatically, dynamically and autonomously **1.** when frame arrives → store MAC address, interface from which frame arrived and current time **2.** if no frames are received from that MAC address within the aging time, address entry is deleted. **Downsides of switch hierarchies** **Lack of traffic isolation:** broadcasts travel entire hierarchy. **Inefficient use of switches:** one large switch for all users does not provide traffic isolation (all traffic would be mixed). **Managing users:** if a user moves between departments, physical cabling needs to be changed. **solution:** VLANs: Multiple virtual LANs defined over single physical LAN infrastructure. **VLAN** **1**multiple virtual LANs defined over a single physical LAN infrastructure **2**port-based vlans, each port is manually assigned to a vlan **3**hosts in a vlan act as if connected to each other via a physical switch **4**each vlan forms a broadcast domain(eg broadcast traffic is not forwarded across) **5**sending traffic between vlans can be done by defining a port as belonging to both vlans and connecting them to a router. **Multiple physical networks:** Connect port belonging to same VLAN on both switches with a cable, **VLAN trunking:** a special port on each switch is configured to interconnect the two vlan switches.

	store-forward	switch/forw table	pros	cons
switch	link-layer	self-learning uses mac addr	plug-and-play high forward-rate	Restricted to spanning-tree topology/arp table grows with number of hosts in subnet/susceptible to broadcast storms
router	network-layer	routing algorithm uses ip addr	Can have redundant paths/firewall protection against lay-2 broadcast storms	
				not plug-and-play larger processing time per packet

Part 6: Video streaming applications and content delivery networks

A video is a sequence of (encoded/compressed) images, Images are displayed at a certain rate (FPS), e.g. 24, 30,... The size of the image (byte) determines the bitrate (Mbps) $\rightarrow \text{Videosize}(b) = \text{bitrate}/8 \text{ (bps)} * \text{duration (s)}$

Dynamic Adaptive Streaming over HTTP (DASH) original video -> segmentations -> encoding
-> Manifest file URL(videosegment, qualitylevel)

```
<---network(HTTP) get/response URL(videosegment, qualitylevel) -->
video client -> decoding -> buffer -> video played
<-> Rate Adaption Heuristic
```

Rate Adaptation Heuristic ("vendor specific", not standardized). Goal: balance between 2 Quality of Experience metrics, it decides which quality to request: 1. video quality should be maximized (and (optionally) fluctuations in quality minimised) & the rebuffering time should be minimized 2. Information that can be used by the client: Buffer level, Estimation of historical bandwidth, Previously requested quality levels

Scalable video streaming using Content Delivery Networks (CDNs): CDN provide scale approach of downloading video (no a single data center but distributed CDNS (The server that is requested is based on lcluster strategies. CDN operation based on DNS interception and redirection. user -> NetCinema -> user -> local DNS for requested vid -> NetCinema auth DNS (forwards to video URL on the KingCDN server) -> local -> KingCDN auth (also starts the KingCDN distributed server) -> local -> then the KingCDN content distribution servers provide user with the file (HTTP) The distributed server that is requested is based on cluster strategies 1. Geographical closeness; Select content server geographically nearest to the local DNS server, Geo-location databases map IP addresses to geographical locations, Disadvantages[Geographic closeness may not reflect network closeness. Some users use remotely located local DNS servers, ignores variations in delay and bandwidth over time] or 2. Real-time measurements; CDN clusters periodically measure delay and loss to local DNS servers using probes (e.g., ping messages), Disadvantages [Local DNS servers may not be configured to respond to probes, Significant overhead to frequently ping all possible local DNS servers]

Case study: Netflix, 1. Uses Amazon Cloud for Netflix website (user management, billing, movie catalogue), Content ingestion (studio master versions), Content processing (transform master into diverse bit rates for different devices), Uploading content versions to private CDN. 2. Netflix private CDN Deployed at > 200 IXP locations (contain full Netflix catalogue), Residential ISPs can deploy a (free) Netflix server rack (contain popular content), Content is pushed during off-peak hours

Quick: A UDP-Based Multiplexed and Secure Transport (Google, May 2021) designed to interface with HTTP/3, quick is also an application layer, Below that UDP (Transport L), Features of Quick and HTTP/3 - "UDP becomes more like TCP"

Connection-oriented and secure, Single handshake that combines session establishment and, authentication/encryption (TCP+TLS separate these) now handshake is 1 RTT